

PRÁCTICA 0

Toma de contacto con el simulador

El objetivo de este documento es familiarizar al alumno con el simulador WinMIPS64. Para conseguirlo se llevarán a cabo varias actividades que irán introduciendo de manera gradual los diferentes aspectos del simulador, desde la edición de programas hasta la ejecución de los mismos.

INSTALACIÓN DEL SIMULADOR.

El simulador únicamente funciona en entornos Microsoft Windows y está disponible, en formato comprimido, en la página de la asignatura.

Una vez descargado es necesario utilizar una herramienta como WinZIP, para poder extraer su contenido, que constará de los siguientes archivos:

- **winmips64.exe** Es el simulador propiamente dicho y la herramienta que utilizaremos en las prácticas.
- **asm.exe** Programa que permite comprobar la sintaxis de un fichero con código MIPS64. Para utilizarlo es necesario una ventana de comandos (cmd.exe), admitiendo como parámetros de entrada el nombre del fichero con extensión ".s". Ejemplo de utilización:
`C:\Winmips64> asm prueba.s` (Suponiendo que tanto **asm** como el fichero **prueba.s** están situados en el mismo directorio)
- **winmipstut.doc** Fichero en formato MS Word con el Manual de Usuario del simulador en inglés (existen traducciones al español en Internet)
- **Ejemplos** Programas de ejemplo listos para ser cargados y ejecutados en el simulador.

EL PROGRAMA.

Antes de mostrar la funcionalidad del simulador se va a describir brevemente el formato de un programa en ensamblador listo para ser cargado en el simulador.

El programa puede editarse con cualquier editor de texto ASCII y consta de dos secciones diferenciadas: definición de variables y código.

En la sección de definición de variables se declaran todas las variables que van a ser usadas por el programa. En realidad la definición de variables no es más que reservar un espacio de memoria para alojarlas y asignarles un nombre simbólico (nombre de la variable) para referenciarlas de una manera más sencilla en el código.

Para definir variables, el ensamblador proporciona una serie de directivas que facilitan la declaración de éstas. Conviene que la declaración de cada variable comience en una nueva línea en su primera columna.

El código (las instrucciones) comienza tras la directiva **".text"** o **".code"**. Las primeras columnas de la línea se reservan para situar una etiqueta como posible destino de una instrucción de salto.

A continuación, a modo de ejemplo, se muestra un pequeño programa que calcula el factorial de un número. Este programa se suministra en el fichero Factorial.s.

```
; Programa que calcula el factorial del numero, mayor
; que cero, almacenado en Num. El calculo se realiza
; mediante iteraciones y el resultado se almacena en Res.

.data

Num:   .byte  4
Res:   .word  0

.text

        lb      r1,Num(r0)
        daddi    r2,r0,1
sigue:  dmul     r2,r2,r1
        daddi    r1,r1,-1
        bnez     r1,sigue
        sd       r2,Res(r0)
        halt
```

En este programa las directivas **".data"** y **".text"** indican el comienzo de la declaración de variables y de código respectivamente.

Las etiquetas **"Num"** y **"Res"**, que preceden a las directivas **".byte"** y **".word"** respectivamente, definen las variables que contienen el número del cual hay

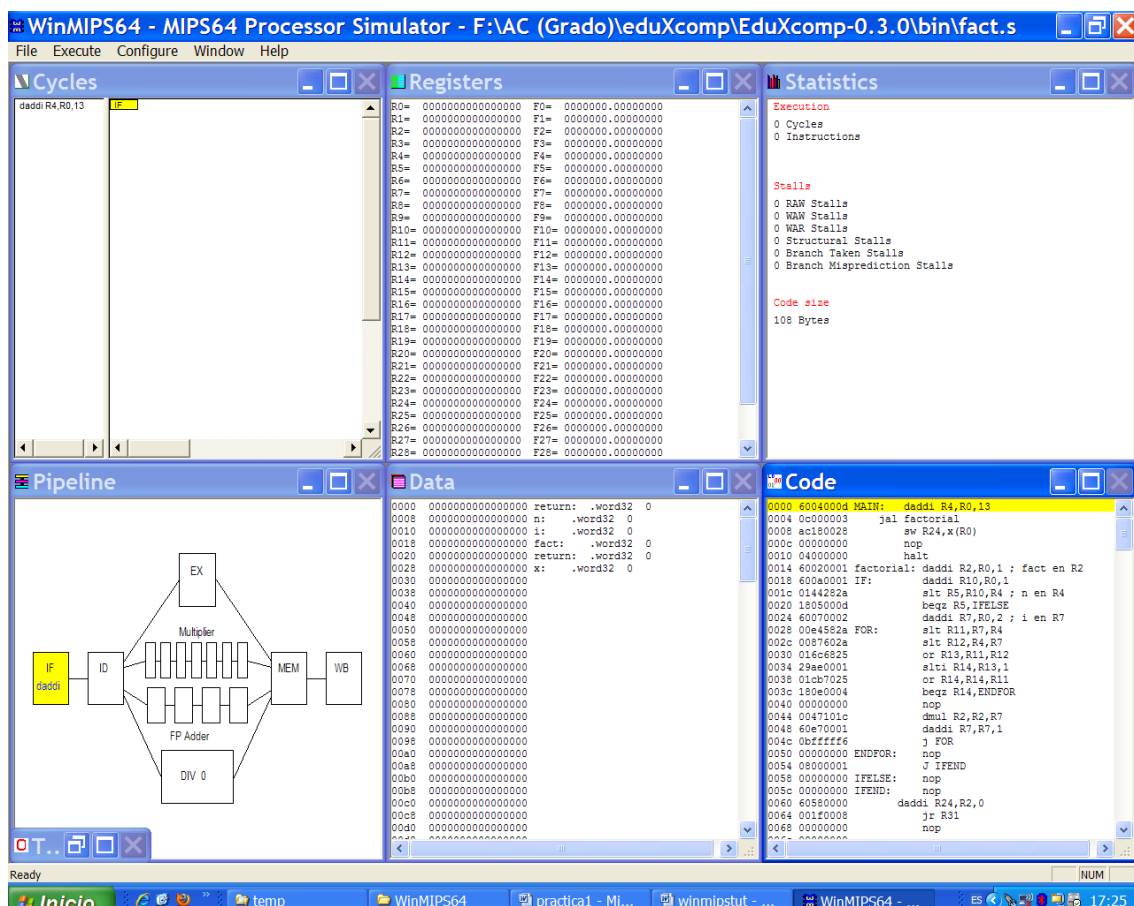
que calcular el factorial y el resultado del mismo. Para el número se ha reservado un solo byte cuyo valor, en este caso, es 4. Para el resultado se ha destinado una palabra de 64 bits que se ha inicializado al valor cero.

El código del programa consta fundamentalmente de un bucle que calcula el factorial del número mediante multiplicaciones sucesivas. Previamente lee el valor del número y al final almacena el resultado del cálculo.

Al principio del código se pueden observar los comentarios que describen la funcionalidad del programa. En general, todo lo que se escribe desde el carácter “;” hasta el final de la línea es un comentario y no tiene efecto sobre el procesador.

ARRANQUE DEL SIMULADOR.

Una vez arrancado el simulador, su aspecto será el siguiente:



Consta de siete ventanas, las seis que podemos ver y otra más que inicialmente está oculta. Su cometido se describe a continuación:

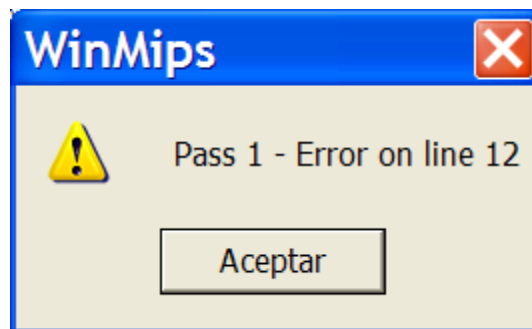
1. **Pipeline** Muestra la estructura del cauce con las etapas y las diferentes unidades funcionales disponibles. Al ejecutarse un programa va mostrando qué instrucciones están en cada etapa.
2. **Code** Muestra el contenido del segmento de código. En la primera columna aparece la dirección de memoria en la que se encuentra cargada la instrucción, a continuación el código máquina y, por último, la instrucción en ensamblador tal y como se editó. Si en el código no se ha indicado nada, el simulador carga el código a partir de la dirección 0 del segmento de código. El contenido de las direcciones se muestra en palabras de 32 bits.
3. **Data** Contenido del segmento de datos con las variables declaradas en el programa. A diferencia de la ventana de código, el contenido de las direcciones se muestra en palabras de 64 bits. El simulador utiliza ordenación **little-endian** para las variables de tamaño superior a un octeto.
4. **Register** Contenido de los registros de propósito general (desde R0 a R31) y de coma flotante (F0-F31). El contenido de los registros puede modificarse en cualquier momento repicando sobre su nombre.
5. **Cycles** Muestra el contenido del cauce a lo largo del tiempo de ejecución. El tamaño del buffer histórico está limitado. Cada etapa del cauce tiene asignado un color y un nombre.
6. **Statistics** Contiene información sobre la ejecución del programa (nº instrucciones ejecutadas, ciclos empleados, paradas clasificadas por tipo de riesgo, etc.).
7. **Terminal** Simula el comportamiento de un terminal de E/S, permitiendo a los programas leer y escribir información desde/hacia el exterior. Esta es la ventana que inicialmente está oculta y que podemos visualizar accediendo a la esquina inferior izquierda del simulador.

Casi todos los valores numéricos mostrados en las anteriores ventanas están expresados en hexadecimal.

En la barra de estado de la ventana principal del simulador se van mostrando informaciones de cómo transcurre la ejecución de un programa en el modo paso a paso (*Single Cycle*)

CARGA DEL PROGRAMA.

Una vez que tengamos editado el programa en un fichero, se procede a su carga en el simulador utilizando la opción Open del menú File. Si tras seleccionar el fichero y dar al botón Abrir no aparece la ventana siguiente es que el programa es sintácticamente correcto y se puede proceder a ejecutarlo:



Una vez cargado, las ventanas de código (*Code*) y datos (*Data*) aparecen rellenas a partir de la información que contenía el fichero que se acaba de cargar.

Cuando aparece la ventana anterior indica que existe al menos un error, pero desgraciadamente muchas veces los programas contienen más de un error siendo deseable corregirlos todos y no necesitar sucesivas cargas del programa en el simulador para detectarlos todos y proceder a corregirlos.

Cómo ya se ha indicado anteriormente junto con el simulador está disponible el programa `asm.exe` que permite comprobar la validez de un programa, mostrando todos los errores que contenga.

EJECUCIÓN DEL PROGRAMA.

La ejecución de un programa puede realizarse de tres formas distintas: Ciclo-a-Ciclo (*Single Cycle*), Multi-Ciclo (*Multi-Cycle*) y Hasta-Breakpoint (*Run-to*).

Ciclo-a-Ciclo permite ir ejecutando el programa comprobando que sucede cada vez que transcurre un nuevo ciclo de reloj. Este modo nos permite ver con gran nivel de detalle el funcionamiento del cauce, desde cuántas instrucciones están activas, riesgos en los que incurren, paradas, etc. Para acceder a este

modo hay que utilizar la opción **Single cycle** del menú **Execute** o utilizar la tecla de función **F7**.

En el modo **Multi-ciclo** la ejecución del programa avanza cinco ciclos (valor por defecto que puede modificarse a través de la opción **Multi-Step** del menú **Configure** o bien pulsando la combinación de teclas **ctrl.+T**). Para acceder a este modo se puede utilizar la opción **Multi-Cycle** del menú **Execute** o bien la tecla de función **F8**.

El modo **Hasta-breakpoint** ejecuta el programa hasta encontrar un punto de parada o hasta el final del programa (normalmente la instrucción **halt** es la última, procediendo a detener el procesador). Este modo está disponible mediante la opción **Run to** del menú **Execute** o la tecla de función **F4**.

Los puntos de parada (*breakpoints*) se fijan repicando con el ratón en una instrucción de la ventana de código, pasando ésta a color azul para indicar que se ha fijado un punto de parada. Para quitarlos se procederá de idéntica forma, es decir, repicando en la instrucción que lo tenga fijado, pasando a continuación del color azul al negro original.

En cualquier momento de la simulación podemos reiniciar el simulador mediante las opciones **Reset MIPS64** y **Full Reset** del menú **File**. La primera opción simula un reset hardware del procesador, la segunda además borra el contenido de la memoria.

PECULIARIDADES DEL SIMULADOR.

El procesador MIPS64 se utiliza en las clases de teoría como vehículo para la explicación de varios conceptos de arquitectura de computadores. Para ello se explica con bastante detalle su funcionamiento la forma en que se previenen los riesgos, las etapas donde se realizan esperas y varios conceptos más.

Sin embargo, el simulador de prácticas a veces se comporta de forma distinta a lo visto en clase. En ocasiones este comportamiento diferente tiene como objetivo forzar situaciones que permiten el estudio de ciertos problemas que se pueden dar en un procesador en *pipeline* que no se dan con el comportamiento del procesador real.

Algunas de estas “peculiaridades” de WinMIPS son las siguientes:

- En WinMIPS64, **el cálculo de la dirección efectiva se realiza en la etapa M** (Memoria), no en la de Ejecución.

- Cuando está activado el “adelantamiento” o *forwarding*, **las instrucciones de multiplicación entregan o adelantan el resultado al final del penúltimo ciclo** (el sexto, M5 en WinMips), no al final del último ciclo.
- Cuando los saltos retardados están habilitados (*Enable Delay Slot* en el Menú de Configuración), **todas las instrucciones de salto**, tanto condicionales como incondicionales **tienen un retardo de un ciclo**, por lo que siempre se ejecuta la instrucción en secuencia inmediatamente siguiente a la del salto.
- En WinMIPS, en general, **las paradas del cauce se realizan en la etapa en la que se produce el problema**, no necesariamente en la etapa de Decodificación (D). Debido a esto, por ejemplo, cuando está activado el adelantamiento, una instrucción aritmética que está pendiente de que le adelanten un operando, puede entrar en la etapa de Ejecución aunque todavía no se lo vayan a proporcionar de manera inmediata, por lo que se empezarían a producir ciclos de retardo RAW de tal instrucción en esta etapa E hasta que le adelanten el dato, y cualquier otra instrucción que le siga en secuencia, podría adelantarla cuando ésta está detenida. Esta situación daría lugar a que se pudieran producir riesgos WAR, cosa que no sería posible si la instrucción se hubiera quedado esperando en su etapa de Decodificación y las instrucciones que le siguen se hubieran quedado bloqueadas en la etapa *Fetch*, tal y como sucede en el procesador MIPS real.

En general, estas diferencias con el procesador real MIPS se irán recordando según vayan apareciendo en las prácticas, y en caso de duda, siempre se puede acudir al profesor.

Conviene tener siempre en cuenta que **el funcionamiento "oficial" es el visto en las clases teóricas** mientras que el comportamiento del simulador quedará relegado exclusivamente a las prácticas.